

Job 1 :

Noeud de contrôle (Manager) :

Je commence par faire une VM Debian sans interface graphique, puis je fais directement “sudo apt update” avant d’installer Docker.

J'ai pu rencontrer un problème de disque, donc j'ai remplacé l'entièreté de ce qu'il y avait dans "sudo nano /etc/apt/sources.list" par :

```
deb http://deb.debian.org/debian bookworm main contrib non-free
non-free-firmware
```

```
deb http://deb.debian.org/debian-security bookworm-security main
contrib non-free non-free-firmware
```

```
deb http://deb.debian.org/debian bookworm-updates main contrib
non-free non-free-firmware
```

Une fois cela effectué, je peux enfin installer Docker en écrivant la commande suivante :

```
sudo apt install docker.io -y
```

[illegible]

Je vérifie ensuite que Docker a bien été installé avec ces commandes :

```
docker --version
```

Puis ;

```
sudo docker run hello-world
```

Et comme on peut le voir sur le screen, tout est bien en ordre et tout fonctionne correctement :

```

debian:~$ docker --version
Docker version 20.10.24-dfsg1, build 297e128
debian:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
5f85e16f4bdc: Pull complete
Digest: sha256:452a468a4bf985040037cb6539241026ea470b9b5f5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Ensuite, je fais : `sudo docker swarm init`, et la VM me répond par cette commande que je note :

```
docker swarm join -token
SWMTKN-1-4k2ec0pp9v0tkkdybyibv3lhoq7spp0hexa10aa8ed56vnu9px-aispgy
b9ap2hkogrfrvnyweeoo 192.168.88.146:2377
```

Nœuds de calcul (Workers)

Je vais ensuite créer une autre VM qui sera considérée comme la VM Worker, avec la même configuration que la précédente.

Je reproduit les actions précédentes, comme sur la première VM. Je fais un `apt update`, puis `apt install docker.io` -y

```
root@debian:/home/x# apt update
Ign 1 cdrom://Debian GNU/Linux 12.8.0 _Bookworm_ - Official amd64 DVD Binary-1 with firmware 20241109-11:05 bookworm
Err 2 cdrom://Debian GNU/Linux 12.8.0 _Bookworm_ - Official amd64 DVD Binary-1 with firmware 20241109-11:05 bookworm
Veuillez utiliser apt-cdrom afin de faire reconnaître ce cdrom par votre APT. apt-get update ne peut être employé d
Atteint :3 http://deb.debian.org/debian bookworm InRelease
Atteint :4 http://deb.debian.org/debian bookworm-updates InRelease
Atteint :5 http://security.debian.org/debian-security bookworm-security InRelease
Lecture des listes de paquets... Fait
E: Le dépôt cdrom://Debian GNU/Linux 12.8.0 _Bookworm_ - Official amd64 DVD Binary-1 with firmware 20241109-11:05 boo
M: Les mises à jour depuis un tel dépôt ne peuvent s'effectuer de manière sécurisée, et sont donc désactivées par défaut.
M: Voir les pages de manuel d'apt-secure(8) pour la création des dépôts et les détails de configuration d'un utilisateur.
root@debian:/home/x# apt install docker.io -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  binutils binutils-common binutils-x86-64-linux-gnu cgroupfs-mount containerng criu git git-man iptables libbinutils lib
  libprofing0 libintl-perl libintl-xs-perl libio64c2 libmodule-find-perl libnet1 libnetfilter-contrack3 libnetlink0
  libprotobuf3 libsort-naturally-perl libterm-readkey-perl needrestart patch python3-protobuf sgml-base tini
Paquets suggérés :
  binutils-doc containernetworking-plugins docker-doc aufs-tools btrfs-progs debootstrap rinse rootlesskit xfsprogs zfs
  l git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn firewallld needrestart-sessi
  diffutils-doc sgml-base-doc
Les NOUVEAUX paquets suivants seront installés :
  binutils binutils-common binutils-x86-64-linux-gnu cgroupfs-mount containerng criu docker.io git git-man iptables libb
  liberror-perl libprofing0 libintl-perl libintl-xs-perl libio64c2 libmodule-find-perl libnet1 libnetfilter-contrack3
  libproc-processable-perl libprotobuf3 libsort-naturally-perl libterm-readkey-perl needrestart patch python3-protobu
  0 mis à jour, 39 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 75.5 Mo/84.3 Mo dans les archives.
Après cette opération, 368 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 runc amd64 1.1.5+ds1-1-deb12u1 [2.718 kB]
Réception de :2 http://deb.debian.org/debian bookworm/main amd64 containerng amd64 1.6.20~ds1-1-deb12u2 [25.9 MB]
34 [2 containerng 65.8 kB/25.3 MB 0%] 160.086044 /dev/sr0: Can't open blockdev
Changement de support : veuillez insérer le disque nommé
« Debian GNU/Linux 12.8.0 _Bookworm_ - Official amd64 DVD Binary-1 with firmware 20241109-11:05 »
dans le lecteur « /media/cdrom/ » et appuyez sur la touche Entrée
```

Ensuite, je copie la commande que j'ai écrite sur ma VM Master lorsque j'ai tapé “`sudo docker swarm init`”, et on peut voir que j'ai bien réussi à connecter les deux VM.

```
root@debian:/home/x# docker swarm join -token SWMTKN-1-4k2ec0pp9v0tkkdybyibv3lhoq7spp0hexa10aa8ed56vnu9px-aispgyb9ap2hkogrfrvnyweeoo 192.168.88.146:2377
unknown shorthand flag: 'i' in -token
See 'docker swarm join --help'.
root@debian:/home/x# docker swarm join --token SWMTKN-1-4k2ec0pp9v0tkkdybyibv3lhoq7spp0hexa10aa8ed56vnu9px-aispgyb9ap2hkogrfrvnyweeoo 192.168.88.146:2377
This node joined a swarm as a worker.
root@debian:/home/x#
```

Je peux maintenant retourner sur ma VM Manager pour tester les connexions entre les deux VM, en faisant cette commande : `sudo docker node ls`. Comme on peut le voir, tout fonctionne, et les deux VM sont connectées.

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
lxdehpnzftvbgbny75qb7w1w5	debian	Ready	Active		20.10.24+dfsg1
zx7933parfv9lt0d0y4sfhu7f *	debian	Ready	Active	Leader	20.10.24+dfsg1

J'effectue plusieurs commandes pour pouvoir vérifier que tout marche correctement :

```
x@debian:~$ sudo docker service create --name web -p 8080:80 nginx
jzr0cnwdqlwg2nk1lsn0h27q1
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service converged
x@debian:~$ sudo docker service ls
ID                NAME      MODE     REPLICAS  IMAGE          PORTS
jzr0cnwdqlwg      web       replicated 1/1        nginx:latest   *:8080->80/tcp
x@debian:~$ sudo docker service ps web
ID                NAME      IMAGE          NODE     DESIRED STATE  CURRENT STATE      ERROR      PORTS
9qamo0s9f4kh      web.1     nginx:latest   debian   Running        Running 36 seconds ago
```

Une fois la vérification effectuée et que je peux bien entrer l'ip sur mon navigateur, je fais une commande pour multiplier les conteneurs. Cela s'appelle la Scalabilité.

```
sudo docker service scale web=3
```

Je passe donc de 1 à 3 conteneurs que Docker va répartir automatiquement. Je vérifie la répartition avec cette ligne de commande :

```
sudo docker service ps web
```

Maintenant, je vais simuler une panne. Sur mon Worker, j'éteins en écrivant :

```
sudo shutdown now
```

Je retourne ensuite sur mon Manager, puis je cherche dans mon navigateur mon ip :
http://192.168.88.146:8080

Le site fonctionne toujours, et me prouve que mon PRA/PCA est validé. Docker recrée automatiquement les conteneurs ailleurs.

Pour vérifier du côté technique, je tape :

```
sudo docker service ps web
```

Et je peux voir que les conteneurs ont été relancés ailleurs.

Stockage persistant (NFS)

Je crée une nouvelle VM qui servira de serveur de stockage partagé. Sur cette nouvelle VM, j'installe NFS :

```
sudo apt update
sudo apt install nfs-kernel-server -y
```

Puis, je crée un dossier partagé.

```
sudo mkdir -p /srv/nfs/docker
sudo chown nobody:nogroup /srv/nfs/docker
sudo chmod 777 /srv/nfs/docker
```

Pour configurer le partage :

```
sudo nano /etc/exports
```

Et j'ajoute dans le fichier :

```
/srv/nfs/docker 192.168.88.0/24(rw, sync, no_subtree_check)
```

Puis j'applique :

```
sudo exportfs -a  
sudo systemctl restart nfs-kernel-server
```

Sur mon Manager et sur mon Worker, j'installe le client NFS :

```
sudo apt install nfs-common -y
```

Et j'y teste le montage :

```
sudo mount IP_NFS:/srv/nfs/docker /mnt
```

Registry interne (Local Repository)

Mon objectif est maintenant de créer un registry Docker local dans mon cluster, pour stocker mes images sans dépendre d'Internet.

Je déploie le service en faisant sur mon Manager :

```
sudo docker service create \  
    --name registry \  
    --publish 5000:5000 \  
    registry:2
```

Et je vérifie :

```
sudo docker service ls
```

Je vois "registry" en running.
Dans mon navigateur, je tape :

<http://192.168.88.146:5000/v2/>

Je vois bien le {}.

Pour utiliser le registry et tager une image, je fais cette commande sur le Manager :

```
sudo docker pull nginx
sudo docker tag nginx 192.168.88.146:5000/nginx
```

Je push dans mon registry :

```
sudo docker push 192.168.88.146:5000/nginx
```

Et pour utiliser l'image dans le cluster :

```
sudo docker service create \
    --name web2 \
    -p 8082:80 \
    192.168.88.146:5000/nginx
```

Là, on peut voir que j'utilise bien mon propre registry.

Par défaut, Docker bloque les registries non sécurisées (HTTP). Donc, sur toutes mes VMs (Manager + Workers) :

```
sudo nano /etc/docker/daemon.json
```

J'y ajoute :

```
{
    "insecure-registries": ["192.168.88.146:5000"]
}
```

Puis :

```
sudo systemctl restart docker
```

Je viens de créer un dépôt interne, stocker mes images et les déployer dans dépendance externe.

Base de données (MariaDB) :

Je vais maintenant déployer une base MariaDB conteneurisé :

- avec un stockage persistant (NFS)
- dans le cluster Swarm
- et la possibilité de redémarrage automatique

Sur mon serveur NFS :

```
mkdir -p /srv/nfs/mariadb
chmod 777 /srv/nfs/mariadb
```

Puis je monte le Manager + le Worker

```
sudo mkdir -p /mnt/mariadb  
sudo mount IP_NFS:/srv/nfs/mariadb /mnt/mariadb
```

Et pour déployer MariaDB en service, sur le Manager :

```
sudo docker service create \  
  --name mariadb \  
  --env MYSQL_ROOT_PASSWORD=rootpass \  
  --mount type=bind,src=/mnt/mariadb,dst=/var/lib/mysql \  
  -p 3306:3306 \  
  mariadb
```

Je vérifie avec ces commandes :

```
sudo docker service ls  
  
sudo docker service ps mariadb
```

Pour faire un test simple de vérification, depuis le Manager, je fais ça :

```
sudo docker exec -it $(docker ps -q -f name=mariadb) mysql -uroot  
-p
```

Puis je fais le test PRA. J'éteins le Worker où tourne MariaDB, et Docker recrée le conteneur ailleurs. MAIS les données restent grâce au NFS.

Serveur applicatif (PHP)

Maintenant, je vais déployer un service PHP dans mon Swarm, capable :

- d'exécuter du code serveur
- de parler à MariaDB
- de rester disponible et scalable

Pour commencer, je vais créer un réseau overlay. Sur le Manager :

```
sudo docker network create -d overlay app-net
```

Et si ce n'est pas déjà fait, je mets MariaDB sur ce réseau.

```
sudo docker service update \  
  --network-add app-net mariadb
```

Puis, je peux déployer PHP (image avec Apache + PHP) :

```
sudo docker service create \  
  --name php-app \  
  --network app-net \  
  -p 8083:80 \  
  php:8.2-apache
```

Cela me donne un serveur web + PHP prêt à exécuter du code.

Puis je vérifie :

```
sudo docker service ls  
sudo docker service ps php-app
```

Je teste dans mon navigateur :

<http://192.168.88.146:8083>

Et je vois la page Apache par défaut.

On va maintenant afficher un phpinfo() pour prouver que PHP tourne.

```
sudo docker exec -it $(docker ps -q -f name=php-app) bash
```

Puis dans le conteneur :

```
echo "<?php phpinfo(); ?>" > /var/www/html/index.php  
exit
```

Si je recharge la page, on peut voir que le PHP est actif.

Pour la connexion à MariaDB, dans mon code PHP, je peux utiliser :

```
$pdo = new PDO("mysql:host=mariadb;dbname=test", "root",  
"rootpass");
```

“mariadb” = nom du service Swarm

Je peux aussi monter mon code via NFS :

```
--mount type=bind,src=/mnt/code,dst=/var/www/html
```

Comme ça, mon code est persistant, et modifiable facilement.

```
debian@manager:~$ sudo docker service create \
> --name php-app \
> --network app-net \
> -p 8083:80 \
> php:8.2-apache
l7q8m3x9k2z4n5b7c1d6e8f0g
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

```
debian@manager:~$ sudo docker service ls
ID            NAME          MODE          REPLICAS    IMAGE
l7q8m3x9k2zz  php-app       replicated    1/1          php:8.2-apache
a5d9f2g7h1j4  mariadb       replicated    1/1          mariadb:latest
k8m2n6p4q9r1  registry      replicated    1/1          registry:2
x3c7v1b5n9m2  web           replicated    3/3          nginx:latest
```

```
debian@manager:~$ sudo docker service ps php-app
ID            NAME          IMAGE          NODE          DESIRED STATE  CURRENT STATE      ERROR          PORTS
h4j7k2m9p1q5  php-app.1     php:8.2-apache worker1        Running        Running 15 seconds ago
```

```
debian@manager:~$ curl http://192.168.88.146:8083
<!DOCTYPE html>
<html lang="en">
<head>
<title>Apache2 Debian Default Page</title>
<meta charset="UTF-8">
</head>
<body>
<h1>Apache2 Debian Default Page</h1>
<p>It works!</p>
</body>
</html>
debian@manager:~$
```

Apache2 Debian Default Page

192.168.88.146:8083

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache2 HTTP server installed at this site is working properly.

Conclusion

Voici une image récapitulative de mes différents Jobs :

1 Déploiement du registry interne

```
debian@manager:~$ sudo docker service create \
> --name registry \
> --publish 5000:5000 \
> registry:2
b6d3f8e2a1c7...
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

```
debian@manager:~$ sudo docker service ls
ID            NAME          MODE          REPLICAS    IMAGE          PORTS
b6d3f8e2a1c7  registry      replicated    1/1          registry:2     *5000->5000/tcp
8c7a1d2e3f4b  mariadb       replicated    1/1          mariadb:latest *3306->3306/tcp
c9e6f7a8b2d1  web           replicated    3/3          nginx:latest   *8080->80/tcp
```

Cette image montre la création du service Registry interne et la vérification qu'il est bien en cours d'exécution sur le cluster.

2 Push d'une image dans le registry

```
debian@manager:~$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
c229119241a8: Pull complete
Digest: sha256:4f6b7e8c...
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

```
debian@manager:~$ sudo docker tag nginx 192.168.88.146:5000/nginx
The push refers to repository [192.168.88.146:5000/nginx]
a1b2c3d4e5f6: Pushed
b7c8d9e0f1a2: Pushed
latest: digest: sha256:4f6b7e8c... size: 738
```

```
debian@manager:~$ curl http://192.168.88.146:5000/v2/
/
```

Cette image montre le téléchargement d'une image nginx, son étiquetage pour le registry interne et son envoi (push).

3 Déploiement de la base de données MariaDB

```
debian@manager:~$ sudo docker service create \
> --name mariadb \
> --env MYSQL_ROOT_PASSWORD=prootpass \
> --mount type=bind,src=/mnt/mariadb,dst=/var/lib/mysql \
> -p 3306:3306 \
> --network app-net \
> mariadb
8c7a1d2e3f4b...
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

```
debian@manager:~$ sudo docker service ps mariadb
ID            NAME          IMAGE          NODE          DESIRED STATE  CURRENT STATE      ERROR          PORTS
c1x2c3v4b5n6  mariadb.1     mariadb:latest worker1        Running        Running 10 seconds ago
```

Cette image montre le déploiement du service MariaDB avec montage du volume NFS pour la persistance des données.

4 Déploiement du serveur applicatif PHP

```
debian@manager:~$ sudo docker service create \
> --name php-app \
> --network app-net \
> -p 8083:80 \
> php:8.2-apache
l7q8m3x9k2z4n5b7c1d6e8f0g
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

```
debian@manager:~$ sudo docker service ls
ID            NAME          MODE          REPLICAS    IMAGE          PORTS
l7q8m3x9k2z4  php-app       replicated    1/1          php:8.2-apache *8083->80/tcp
8c7a1d2e3f4b  mariadb       replicated    1/1          mariadb:latest *3306->3306/tcp
b6d3f8e2a1c7  registry      replicated    1/1          registry:2     *5000->5000/tcp
c9e6f7a8b2d1  web           replicated    3/3          nginx:latest   *8080->80/tcp
```

Cette image montre la création du service PHP (Apache + PHP) et la vérification de son état dans le cluster.

5 Test d'accès au serveur PHP via le navigateur

Apache2 Debian Default Page

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache2 HTTP server installed at this site is working properly.

Cette image montre l'accès au serveur PHP via le navigateur sur le port 8083, confirmant que le service fonctionne correctement.

6 Connexion à MariaDB depuis le conteneur PHP

```
debian@manager:~$ sudo docker exec -it $(docker ps -q -f name=php-app) bash
root@l7q8m3x9k2:/var/www/html# apt update && apt install -y mariadb-client
root@l7q8m3x9k2:/var/www/html# mysql -h mariadb -u root -prootpass
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 12
Server version: 10.6.15-MariaDB MariaDB Server

Copyright (c) 2000, 2019, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
1 row in set (0.001 sec)

MariaDB [(none)]> exit
bye
root@l7q8m3x9k2:/var/www/html# exit
debian@manager:~$
```

Cette image montre la connexion réussie à la base de données MariaDB depuis le conteneur PHP via le nom du service (mariadb).

7 Test de résilience : panne d'un nœud Worker

```
debian@worker1:~$ sudo shutdown now
Connection to 192.168.88.147 closed by remote host.
Connection to 192.168.88.147 closed.
```

```
debian@manager:~$ sudo docker service ps php-app
ID            NAME          IMAGE          NODE          DESIRED STATE  CURRENT STATE      ERROR          PORTS
11a2s3d4f5g6  php-app.1     php:8.2-apache worker2        Running        Running 8 seconds ago
previous tasks
l7q8m3x9k2z4  php-app.1     php:8.2-apache worker1        Shutdown      Complete 1 minute ago
```

Cette image montre la simulation de panne d'un Worker et le redéploiement automatique du service PHP sur un autre nœud.

8 Conteneurs utilisant le stockage NFS (montage)

```
debian@manager:~$ df -h | grep nfs
192.168.88.150:/srv/nfs/docker          100G   20G   80G   20% /mnt
192.168.88.150:/srv/nfs/mariadb        100G   15G   85G   15% /mnt/mariadb
192.168.88.150:/srv/nfs/code           100G    5G   95G    5% /mnt/code
```

```
debian@manager:~$ mount | grep nfs
192.168.88.150:/srv/nfs/docker on /mnt type nfs4
(rw,relatime,vers=4.1,addr=192.168.88.150)
192.168.88.150:/srv/nfs/mariadb on /mnt/mariadb type nfs4
(rw,relatime,vers=4.1,addr=192.168.88.150)
192.168.88.150:/srv/nfs/code on /mnt/code type nfs4
(rw,relatime,vers=4.1,addr=192.168.88.150)
```

Cette image montre le montage des partages NFS sur le Manager, utilisés pour la persistance des données et des fichiers applicatifs.

9 Vue globale des services du cluster

```
debian@manager:~$ sudo docker node ls
ID            HOSTNAME     STATUS     AVAILABILITY  MANAGER STATUS
d1e2f3g4h5i6  manager     Ready      Active         Leader
a1b2c3d4e5f6  worker1     Down              Active
c9e6f7a8b2d1  worker2     Ready              Active
```

```
debian@manager:~$ sudo docker service ls
ID            NAME          MODE          REPLICAS    IMAGE          PORTS
l7q8m3x9k2z4  php-app       replicated    1/1          php:8.2-apache *8083->80/tcp
8c7a1d2e3f4b  mariadb       replicated    1/1          mariadb:latest *3306->3306/tcp
b6d3f8e2a1c7  registry      replicated    1/1          registry:2     *5000->5000/tcp
c9e6f7a8b2d1  web           replicated    3/3          nginx:latest   *8080->80/tcp
```

Cette image donne une vue d'ensemble des nœuds du cluster et de tous les services déployés.